Rapport de réalisation — Sprint 2

Fitness Franchise

Projet universitaire de développement logiciel Version de travail pour relecture — 1er novembre 2026

Information	À compléter ou confirmer
Établissement / formation	[À compléter]
Module	[À compléter]
Équipe	Membres de l equipe [répartition à confirmer]
Encadrant / Product Owner	[À compléter]
Période du sprint	[À compléter]
État du code décrit	Dépôt local au 1er novembre 2026 ; plusieurs évolutions Sprint 2 ne sont pas encore publiées sur GitHub

> **Note de version.** Ce rapport décrit le code local examiné et les essais rapportés par l'équipe. Le dernier commit publié observé sur `main` est `af4fd06` (« feat: complete Fitness Premium redesign phases 1 and 2 »). Le répertoire local contient des modifications non commitées liées au Sprint 2 : elles ne doivent pas être présentées comme déjà publiées sur GitHub. Les résultats des essais manuels décrits ci-dessous sont ceux communiqués par l'équipe ; les essais automatisés sont identifiés séparément.

Sommaire

1. Résumé du sprint
2. Contexte, objectifs et périmètre
3. Besoins fonctionnels
4. Architecture et technologies
5. Données et sécurité
6. Fonctionnalités réalisées
7. Stratégie de test et résultats
8. Difficultés rencontrées
9. Bilan, limites et perspectives
10. Captures et annexes à compléter

1. Résumé du sprint

Le Sprint 2 a pour objectif de permettre à un adhérent de se connecter à Fitness Franchise, de consulter ses entraînements et de suivre sa progression. Le périmètre a évolué au cours du sprint : il comprend désormais le profil, un catalogue de vingt équipements, la création et le filtrage des séances, les statistiques, un objectif hebdomadaire et un simulateur de tapis de course.

L'application associe une interface React à une API Spring Boot et à une base PostgreSQL. Un simulateur Python peut également s'authentifier auprès de l'API et enregistrer une séance. Les activités sont rattachées au compte connecté ; les équipements qui ne mesurent pas la distance peuvent être enregistrés avec une distance absente (`NULL`).

Les tests automatisés couvrent les principaux contrats frontend, backend et simulateur. L'équipe a en outre confirmé plusieurs essais réels : le simulateur a créé une séance de tapis de 16 minutes et 2,74 km, acceptée par l'API en HTTP 202 et retrouvée dans PostgreSQL avec `equipment_id = 3`. Des séances de rameur et de presse à cuisses ont aussi été enregistrées et vérifiées. Les filtres par date et équipement, ainsi que l'affichage des distances non applicables, ont été testés manuellement.

2. Contexte, objectifs et périmètre

Fitness Franchise est une application de suivi de l'activité sportive. Le Sprint 2 complète le parcours commencé lors du sprint précédent en donnant à l'adhérent un espace personnel et des outils de suivi.

2.1 Objectifs

- Fournir une authentification par compte et protéger les actions personnelles.
- Permettre à l'adhérent de consulter et de mettre à jour son profil.
- Enregistrer des séances sur les équipements du catalogue.
- Afficher l'historique personnel, ses filtres et ses détails.
- Présenter des indicateurs de progression et un graphique hebdomadaire.
- Définir un objectif de fréquence d'entraînement pour la semaine.
- Permettre au simulateur Python de créer une séance de tapis par la même API.

2.2 Périmètre et exclusions

Le Sprint 2 porte sur le suivi individuel des entraînements. Il ne met pas en œuvre un système réel de réservation, de disponibilité par salle, d'administration de franchise, de badges ou d'export CSV. Les photographies du catalogue public sont des visuels d'illustration et ne prouvent ni l'existence physique ni la disponibilité d'un équipement.

3. Besoins fonctionnels

Référence	Besoin	Règle ou résultat attendu
BF-01	Créer un compte et se connecter	Les actions personnelles nécessitent une session authentifiée.
BF-02	Consulter et modifier son profil	Ville, genre et objectif sportif sont facultatifs et validés.
BF-03	Consulter le catalogue utile aux séances	L'API renvoie les équipements avec leur identifiant, leur nom, leur type et leur catégorie.
BF-04	Enregistrer une séance	La date et une durée positive sont requises ; l'équipement est référencé par `equipmentId`.
BF-05	Gérer la distance selon l'équipement	Une distance est requise pour tapis, vélo, elliptique et rameur ; elle est absente pour les autres machines.
BF-06	Consulter et filtrer ses séances	Les filtres de dates et de type s'appliquent aux données personnelles ; la date de fin est incluse.
BF-07	Suivre son activité	Les séances et durées sont comptées même lorsqu'aucune distance n'est mesurée.
BF-08	Définir un objectif hebdomadaire	La cible est un entier compris entre 1 et 14 séances.
BF-09	Simuler une séance de tapis	Le simulateur s'authentifie, identifie le tapis dans le catalogue et envoie la séance à l'API.

4. Architecture et technologies

4.1 Composants

Composant	Technologie	Rôle
Interface web	React, Vite, JavaScript et CSS	Formulaires, espace adhérent, historique, filtres, statistiques et objectif.
API	Java, Spring Boot, Spring Security et validation Jakarta	Authentification, règles métier, contrôle des requêtes et accès aux données.
Base de données	PostgreSQL	Comptes, équipements, activités, objectifs et mesures associées.
Simulateur	Python 3, bibliothèque standard	Simulation et envoi d'une séance tapis par HTTP.
Tests	JUnit / tests Spring, Node, tests Python `unittest`	Vérification des contrats, règles métier et comportements simulés.

4.2 Flux principal

1. L'adhérent ouvre une session depuis l'interface React.
2. Le navigateur conserve la session et inclut ses cookies aux appels API.
3. Pour les écritures, l'interface demande un jeton CSRF et le transmet avec la requête.
4. Spring Boot valide la requête et détermine l'adhérent à partir de la session.
5. Les données sont lues ou enregistrées dans PostgreSQL.
6. L'interface récupère les informations personnelles et les présente à l'adhérent.

Le simulateur utilise aussi l'API authentifiée. Il ne reçoit pas de cookie depuis le navigateur de l'utilisateur : il ouvre sa propre session HTTP pendant son exécution.

4.3 API principales

Méthode et route	Utilisation
`GET /api/auth/csrf`	Initialiser la session et obtenir un jeton CSRF.
`POST /api/auth/register`	Créer un compte.
`POST /api/auth/login` / `POST /api/auth/logout`	Ouvrir ou fermer une session.
`GET /api/me` / `PUT /api/me`	Lire ou mettre à jour le profil connecté.
`GET /api/equipments`	Charger le catalogue authentifié.
`GET /api/activities` / `POST /api/activities`	Lire l'historique ou soumettre une activité.
`GET /api/me/activity-summary`	Obtenir les totaux de la période sélectionnée.

Méthode et route	Utilisation
`GET /api/me/activity-trend`	Obtenir le nombre de séances par semaine.
`GET /api/me/weekly-goal` / `PUT /api/me/weekly-goal`	Lire ou modifier l'objectif de la semaine.
`GET /api/health`	Vérifier la disponibilité générale de l'API.

Les lectures de séances acceptent notamment `from`, `to` et `equipmentType`. L'API interprète la fin de période de manière inclusive, selon la convention Europe/Paris.

5. Données et sécurité

5.1 Modèle de données

Le modèle utilise notamment les entités `adherents`, `equipments`, `activities`, `weekly_goals` et `activity_stats`. Une activité peut référencer l'équipement par `equipment_id` tout en conservant `equipment_type` pour les anciennes données et la compatibilité des clients existants. Les liens vers un adhérent et un équipement peuvent rester absents sur des lignes anciennes afin de préserver l'historique antérieur.

La colonne `activities.distance_km` accepte maintenant `NULL`. Selon les validations communiquées par l'équipe, cette évolution a été appliquée dans PostgreSQL et les séances de rameur et de presse à cuisses ont été vérifiées. Dans l'application, `NULL` signifie « distance non applicable » : ce n'est pas une distance de zéro kilomètre.

Le catalogue comporte vingt équipements, répartis en quatre catégories existantes, cinq équipements par catégorie. Leur insertion dans la base a été réalisée par l'équipe. La migration des champs de profil (`city`, `gender` et `sport_objective`) est fournie dans le dépôt ; l'état de son application sur la base de démonstration utilisée pour le rapport reste à confirmer.

5.2 Protection des comptes et des séances

- L'authentification repose sur une session gérée par Spring Security.
- Les mots de passe ne sont pas affichés dans les journaux ; ils sont stockés sous forme hachée par le backend.
- Le jeton CSRF protège les requêtes d'écriture.
- Le serveur choisit le propriétaire d'une activité depuis l'identité authentifiée, et non depuis un identifiant fourni par le navigateur.
- Les listes, statistiques et objectifs sont propres au compte connecté.
- Le simulateur masque la saisie du mot de passe et n'affiche ni identifiants, ni cookies, ni jeton de session dans ses journaux.

6. Fonctionnalités réalisées

6.1 Authentification et profil adhérent

L'application propose l'inscription, la connexion et la déconnexion. Après connexion, le profil est chargé depuis l'API. Le formulaire de profil permet de consulter et de modifier les champs facultatifs ville, genre et objectif sportif, avec validation, messages de retour et possibilité d'annuler les modifications.

La photo de profil, lorsqu'elle est utilisée dans l'interface, est compressée côté navigateur et conservée localement dans le navigateur. Elle n'est pas envoyée à l'API ni stockée dans PostgreSQL ; elle ne doit donc pas être présentée comme une photo synchronisée entre appareils.

Validation disponible : tests backend de lecture et mise à jour du profil, session obligatoire, CSRF, validation des valeurs et contrôle de l'identité utilisée. Le formulaire actuel doit encore être photographié pour le rapport. L'application de la migration des trois champs sur la base de démonstration est à confirmer avant une démonstration avec persistance réelle.

> **Figure 1 — Profil adhérent.** *Emplacement à compléter : capture actuelle du profil, avec les champs facultatifs et le message de confirmation. Utiliser un compte fictif et masquer l'adresse électronique.*

6.2 Catalogue des équipements

L'endpoint authentifié `GET /api/equipments` sert au formulaire d'ajout de séance. La réponse contient les champs nécessaires à la sélection : identifiant, nom, type et catégorie. Le formulaire charge ainsi les vingt machines stockées dans PostgreSQL au lieu de s'appuyer sur une courte liste codée en dur.

Il faut distinguer ce catalogue opérationnel pour la saisie des séances de la page publique de présentation. Cette dernière comporte encore des données de démonstration, des statuts fictifs et une mention indiquant que les réservations réelles ne sont pas disponibles. Elle ne constitue pas un inventaire de disponibilité par salle.

Validation disponible : tests du contrôleur et du service de catalogue ; chargement réel des vingt équipements confirmé dans les échanges avec l'équipe. La capture publique existante du catalogue concerne un aperçu fictif et ne doit pas servir de preuve des vingt équipements réels.

> **Figure 2 — Catalogue utilisé dans « Ajouter une séance ».** *Emplacement à compléter : montrer le sélecteur alimenté par l'API et les quatre catégories, sans présenter les visuels fictifs comme l'inventaire réel.*

6.3 Ajout et enregistrement des séances

Le formulaire demande une date, une durée et un équipement. Le frontend envoie `equipmentId`. Le backend vérifie que l'équipement existe, conserve son identifiant et son type historique, puis enregistre l'activité pour le compte authentifié.

La distance dépend du type :

- **Tapis de course (`TREADMILL`), vélo (`BIKE`), elliptique (`ELLIPTICAL`) et rameur (`ROWING`) :** distance obligatoire.
- **Les seize autres équipements :** distance non applicable ; la valeur stockée est `NULL`.

Cette règle permet de saisir les équipements de musculation sans leur attribuer une distance fictive. Les séances historiques restent conservées. Pour les statistiques, une activité sans distance contribue au nombre de séances et à la durée, mais pas à un kilométrage réel.

Validations manuelles confirmées par l'équipe :

- Rameur : séance créée et retrouvée dans PostgreSQL.
- Presse à cuisses : séance créée et retrouvée avec `distance_km = NULL`.
- Tapis simulé : durée de 16 minutes, distance de 2,74 km, réponse HTTP 202 et ligne vérifiée dans PostgreSQL avec `equipment_id = 3`.

> **Figure 3 — Formulaire cardio.** *Emplacement à compléter : équipement cardio sélectionné et champ distance obligatoire visible.*
>
> **Figure 4 — Formulaire de renforcement.** *Emplacement à compléter : presse ou autre machine de renforcement sélectionnée, sans champ distance.*
>
> **Figure 5 — Séance non mesurée en distance.** *Emplacement à compléter : ligne ou détail d'une séance affichant « Non applicable », et non « 0 km ».*

6.4 Historique, filtres et détails

L'adhérent consulte la liste de ses propres séances, triées de la plus récente à la plus ancienne. Il peut filtrer par dates et par type d'équipement. La date de fin choisie est incluse ; les calculs de période prennent en compte le fuseau Europe/Paris. Les totaux visibles sur la liste correspondent aux séances filtrées.

Un détail de séance présente les informations disponibles. La distance et la vitesse moyenne ne doivent pas être interprétées comme mesurées lorsque la distance est `NULL`. Les anciennes activités sans lien vers un compte ne sont pas exposées dans l'espace personnel.

Validation disponible : tests backend des filtres, des bornes de dates et de l'isolation par utilisateur ; l'équipe confirme avoir testé manuellement les filtres par date et par équipement ainsi que l'affichage des séances sans distance.

> **Figure 6 — Filtres et journal des séances.** *Emplacement à compléter : période et équipement sélectionnés, résultats personnels et totaux.*
>
> **Figure 7 — Détail d'une séance sans distance.** *Emplacement à compléter : le libellé doit indiquer que la distance est non applicable.*

6.5 Statistiques et tableau de bord

La page de statistiques présente les totaux de séances, de durée et de distance pour la période et le type choisis, ainsi qu'un graphique hebdomadaire. Les filtres sont transmis aux endpoints de synthèse et de tendance. Le nombre de séances et leur durée incluent les activités sans distance ; le total de distance n'additionne que les valeurs mesurées.

Le graphique représente les semaines de la période, y compris les semaines vides ou partielles. Les semaines sont calculées selon la convention Europe/Paris. Le tableau de bord n'est donc pas limité aux séances ayant une distance.

Validation disponible : tests des calculs de synthèse et de tendance, des semaines vides et partielles, des périodes et des distances nulles. La capture actuelle de ces vues reste à fournir, car les images historiques du dépôt ne montrent pas les derniers changements.

> **Figure 8 — Statistiques filtrées.** *Emplacement à compléter : filtres, cartes de synthèse et période cohérente.*
>
> **Figure 9 — Activité hebdomadaire.** *Emplacement à compléter : graphique et légende lisible des séances par semaine.*

6.6 Objectif hebdomadaire

Un adhérent peut définir une cible de une à quatorze séances pour la semaine courante. L'API conserve un objectif par compte et par semaine. La progression compare les séances réalisées à cette cible ; elle est plafonnée à 100 % et affiche « Objectif atteint » lorsque la cible est atteinte ou dépassée.

La semaine commence le lundi et se termine avant le lundi suivant, selon le fuseau Europe/Paris. Modifier la cible d'une semaine ne remplace pas les objectifs des semaines précédentes.

Validation disponible : tests du service et de l'API couvrant les limites, la progression, le dépassement, les semaines et l'isolation des comptes. Une capture récente de la progression dans l'interface doit être ajoutée.

> **Figure 10 — Progression hebdomadaire.** *Emplacement à compléter : cible, nombre de séances réalisées et barre de progression.*
>
> **Figure 11 — Objectif atteint.** *Emplacement à compléter : état atteint et confirmation d'enregistrement.*

6.7 Simulateur Python

Le simulateur conserve la génération d'une durée, d'une vitesse et d'une distance. Il s'authentifie auprès du backend, gère sa session et le jeton CSRF, appelle ensuite `GET /api/equipments`, cherche le type exact `TREADMILL`, puis envoie `equipmentId` et `equipmentType: "TREADMILL"` à `POST /api/activities`.

Si le catalogue échoue ou ne contient pas de tapis, le programme s'arrête avant l'enregistrement. La saisie du mot de passe est masquée et les journaux ne doivent pas contenir d'identifiants, de cookies ou de secrets.

Validation réelle rapportée par l'équipe : une séance de tapis de 16 minutes et 2,74 km a été acceptée en HTTP 202, puis vérifiée dans PostgreSQL avec `equipment_id = 3`. Les tests automatisés du simulateur utilisent des réponses HTTP simulées ; ils ne remplacent pas cette vérification réelle.

> **Figure 12 — Exécution réelle du simulateur.** *Emplacement à compléter : terminal VS Code montrant la réussite et le statut HTTP 202, après masquage de tout identifiant ou donnée de compte.*
>
> **Figure 13 — Vérification PostgreSQL.** *Emplacement à compléter : résultat en lecture seule montrant la durée, la distance et `equipment_id`, sans afficher l'identifiant de l'adhérent.*

7. Stratégie de test et résultats

7.1 Résultats automatisés connus

Niveau	Résultat connu	Portée et réserve
Backend Spring Boot	109 tests exécutés, 0 échec, 0 erreur, 7 ignorés.	Les tests couvrent les règles métier, l'API, la sécurité, les filtres, les statistiques et les équipements. Les tests PostgreSQL soumis à activation explicite sont notamment ignorés dans cette exécution.
Intégration PostgreSQL isolée	Des tests d'intégration antérieurs ont été rapportés comme réussis sur une base isolée.	Ils vérifient notamment l'authentification et l'association/propriété des séances. Ils ne prouvent pas à eux seuls l'état de la base de démonstration actuelle.
Frontend	Build Vite réussi, lint Oxlint réussi et 51 tests Node réussis.	Ces résultats couvrent le code frontend contrôlé, mais ne constituent pas une recette visuelle authentifiée complète.
Simulateur Python	7 tests avec réponses HTTP simulées réussis ; syntaxe vérifiée.	Aucun accès réseau ni écriture PostgreSQL pendant ces tests automatisés.
Essais manuels communiqués	Tapis simulé, rameur, presse à cuisses, filtres par dates et équipement, affichage de distance non applicable.	Résultats déclarés par l'équipe et non rejoués dans cette analyse.

7.2 Lecture des résultats

Les tests automatisés démontrent la validation des contrats et de plusieurs règles de sécurité. Les essais manuels complètent cette couverture en confirmant les échanges réels avec PostgreSQL pour les trois types de séance cités. Les tests ignorés et l'absence de capture récente des parcours authentifiés doivent rester visibles dans le bilan : ils ne doivent pas être remplacés par une affirmation générale de validation complète.

Le point de santé `GET /api/health` vérifie la disponibilité de l'API ; il ne suffit pas à démontrer que l'accès PostgreSQL ou l'ensemble des opérations du Sprint 2 fonctionne.

8. Difficultés rencontrées

1. **Configuration de l'environnement Java.** Le backend nécessite un JDK accessible par `JAVA_HOME` et un démarrage Spring Boot correct.
2. **Connexion PostgreSQL.** Des problèmes de connexion et d'identifiants ont dû être distingués des erreurs de schéma. Les secrets ne sont pas consignés dans ce rapport.
3. **Évolution prudente du schéma.** L'ajout de champs de profil et de clés étrangères devait préserver les données existantes. Les migrations sont séparées ; leur application ne doit pas être supposée sans vérification de la base cible.
4. **Distance non pertinente pour certaines machines.** La colonne initialement obligatoire ne permettait pas de représenter les séances de musculation. Elle a été rendue nullable selon la confirmation de l'équipe, puis la règle a été appliquée dans l'API, l'interface et l'affichage des statistiques.
5. **Session du navigateur partagé.** Un appel d'API authentifiée avait renvoyé HTTP 401 lorsque la session n'était pas transmise. Le test d'API doit se faire dans un contexte authentifié ; aucun cookie de session n'est copié ni demandé.
6. **Version du backend en cours d'exécution.** Un serveur déjà démarré peut ne pas contenir un nouvel endpoint ; le redémarrage doit éviter de lancer un deuxième serveur sur le même port.
7. **Documentation et code à synchroniser.** Certains documents et captures décrivent des étapes antérieures. Le code local contient également des évolutions non publiées ; le rapport distingue donc les faits observés, les résultats rapportés et les éléments restant à confirmer.

9. Bilan, limites et perspectives

9.1 Bilan

Le Sprint 2 fournit un parcours cohérent de suivi individuel : authentification, profil, saisie de séances sur les équipements, historique filtrable, statistiques et objectif hebdomadaire. Le simulateur utilise la même API que l'interface. Les règles de propriété des données restent contrôlées côté serveur. Les séances sans distance sont conservées sans leur attribuer de kilométrage artificiel.

9.2 Limites actuelles

- Plusieurs adaptations locales n'apparaissent pas encore dans le dernier commit GitHub observé.
- Sept tests backend étaient ignorés dans la suite complète connue ; les tests d'intégration doivent être rejoués dans un environnement PostgreSQL isolé et documenté.
- Les captures existantes dans `docs/assets/sprint2/` datent du 30 octobre 2026 et illustrent un jeu fictif ou un état antérieur. Elles ne prouvent pas les formulaires récents, la règle de distance `NULL` ni les résultats manuels communiqués.
- La page publique des équipements présente des modèles et disponibilités de démonstration ; les réservations réelles et l'inventaire par salle ne sont pas réalisés.
- La photo de profil est locale au navigateur.
- L'application de la migration des champs facultatifs de profil sur la base choisie pour la démonstration reste à confirmer.
- Le responsive a fait l'objet d'un aperçu à fenêtre étroite dans les anciennes captures, mais cela ne vaut pas une recette sur téléphone réel.

9.3 Perspectives

1. Rejouer les tests d'intégration PostgreSQL sur une base de test isolée et noter les versions et commandes utilisées.
2. Confirmer l'état des migrations sur la base de démonstration, sans réexécuter une migration à l'aveugle.
3. Actualiser les captures depuis une session de test et harmoniser les documents du dépôt avec le code final.
4. Publier les changements locaux après revue et validation de l'équipe.
5. Traiter séparément les fonctionnalités hors périmètre — réservation persistante, inventaire réel, badges ou export — si elles sont retenues pour un sprint futur.

10. Captures et annexes à compléter

10.1 Captures à fournir pour la version relue du rapport

Les pages personnelles ne sont pas accessibles dans le navigateur partagé actuel sans session authentifiée. Aucune session ni aucun cookie n'a été manipulé. Pour obtenir des preuves visuelles, l'équipe peut prendre les captures suivantes avec un compte fictif :

N°	Capture nécessaire	Consignes
1	Profil actuel avec ville, genre et objectif sportif	Montrer la sauvegarde réussie ; masquer email, identifiant et toute donnée personnelle.
2	Catalogue réel dans le formulaire de séance	Montrer les vingt entrées ou les quatre catégories, sans confondre ce sélecteur avec l'aperçu public.
3	Formulaire avec un équipement cardio	Montrer la distance obligatoire.
4	Formulaire avec une machine de renforcement	Montrer que le champ de distance est absent.
5	Historique avec filtres par dates et équipement	Montrer les critères et les résultats du compte fictif.
6	Séance sans distance dans le journal ou son détail	Le libellé doit être « Non applicable » (ou équivalent), pas `0 km`.
7	Statistiques et graphique actuels	Vérifier que les filtres et les valeurs concernent la même période.
8	Objectif en progression et objectif atteint	Capturer ces deux états si possible.
9	Terminal du simulateur réel	Montrer la réussite HTTP 202, sans mot de passe, email, cookie ou identifiant de session.
10	Résultat pgAdmin en lecture seule	Montrer le tapis, le rameur et la presse avec `equipment_id` et la distance appropriée ; masquer `adherent_id` et toute donnée personnelle.

Ne pas relancer le simulateur uniquement pour obtenir une capture sans accord de l'équipe : une exécution réussie crée une nouvelle séance persistante. Une requête de consultation PostgreSQL suffit pour illustrer une séance déjà enregistrée.

10.2 Captures déjà présentes dans le dépôt

Le dossier assets/sprint2 contient seize captures réelles prises le 30 octobre 2026. Elles documentent l'accueil, l'inscription, la connexion, un profil de démonstration, des écrans d'activité, les statistiques, l'objectif et un aperçu d'équipements. Elles sont utiles comme références de l'état précédent, mais doivent être contrôlées avant insertion dans le PDF :

- `05-profil.png` ne montre pas le formulaire actuel avec les champs facultatifs.

- `14-equipements-demo.png` et `15-fiche-equipement-demo.png` montrent des données de démonstration, pas l'inventaire PostgreSQL.
- Les captures d'activités, statistiques et objectifs reposent sur un ancien jeu de démonstration et ne démontrent pas les derniers comportements concernant `distance_km = NULL`.
- `16-accueil-fenetre-etroite.png` montre une fenêtre étroite ; ce n'est pas un test sur téléphone réel.

Pour la version finale, remplacer ou légender explicitement toute capture qui ne correspond pas au code et aux essais décrits dans ce rapport. Ne pas utiliser d'image générée comme preuve fonctionnelle.

10.3 Annexes techniques possibles

- Tableau des routes API et des codes de réponse attendus.
- Extrait anonymisé du rapport des tests automatisés.
- Schéma simplifié des entités et de leurs relations.
- Requête PostgreSQL de consultation en lecture seule utilisée pour vérifier les séances existantes.
- Procédure de lancement du simulateur décrite dans simulator/README.md.

Fin du rapport de travail — version Markdown à relire avant la mise en page PDF.